Séquence 2

2

TP guidé — Brancher et piloter la connexion

Vous utilisez le projet. Consultez le corrigé après vos essais. Les six TODO se trouvent dans **frontend/src/useConnection.js**. Le backend et l'interface sont fournis.

Préparer le poste

Le poste Node 24 et les outils ont été préparés en S1. Arrêtez les anciens services pour libérer 3000 et 5173. Ouvrez un terminal dans projet, puis :

```
npm ci
npm run dev:back
```

Dans un second terminal, dans le même dossier racine :

```
npm run dev:front
```

Ouvrez **http://127.0.0.1:5173**. Le badge indique « Non connecté ». Au départ, Connecter affiche une consigne : le premier TODO contient un **return** temporaire qu'il faudra remplacer.

Partie A — Connecter le navigateur

TODO 1 · Instancier le client

Dans le try, remplacez **les deux lignes temporaires** :

```
information.value = 'TODO 1 : instancier WebSocket dans useConnection.js.';
return;
```

par la construction native affectée à **current**, en utilisant **url.value** comme adresse. Ne créez pas un client Socket.IO. Ne mettez pas **await** devant le constructeur.

Résultat intermédiaire : le navigateur peut tenter une ouverture, même si votre indicateur n'est pas encore synchronisé. Les autres TODO restent nécessaires.

TODO 2 · Reporter l'état initial

Après **socket = current**, affectez l'état courant du nouvel objet à **readyState.value**.

Indice : le nom de la propriété est le même sur l'objet natif, mais la donnée Vue utilise .value.

Résultat attendu : une ligne construction avec l'état 0 apparaît. Cette ligne ne prouve pas encore l'ouverture.

TODO 3 · Traiter l'ouverture

Dans **onOpen()** :

1. Recopiez l'état courant de **current** dans **readyState.value**.
2. Affectez à **information.value** un texte signalant l'ouverture.
3. Conservez l'appel **record('open')** fourni.

Résultat attendu : le badge indique « Connecté », l'état observé vaut 1 et le journal contient open. Le bouton Connecter devient indisponible et Déconnecter disponible. Il ne fermera réellement la connexion qu'après TODO 6.

TODO 4 · Traiter une erreur

Dans **onError()**, renseignez un texte qui invite à vérifier l'adresse et le serveur. Ne prétendez pas connaître la cause exacte et n'ajoutez pas une valeur numérique inventée pour **ERROR**.

Le gestionnaire **close** sera terminé dans la partie B. Tant qu'il manque, un échec peut laisser un badge incomplet : c'est une étape de réalisation, pas le comportement final attendu.

Vérification réseau ciblée

Après sauvegarde, si nécessaire rechargez la page pour repartir sans tentative active. Ouvrez Réseau dans les outils du navigateur, puis cliquez sur Connecter. Sélectionnez la connexion vers **127.0.0.1:3000/chat**.

Relevez :

URL de la connexion métier :
Statut de l'ouverture HTTP/1.1 :
En-tête signalant le protocole demandé :
Événement ajouté dans le journal :

Point d'étape

Expliquez pourquoi le texte « Connecté » se trouve dans **onOpen()** plutôt que juste après le constructeur. Si le navigateur ne se connecte pas, vérifiez **/health**, l'adresse 3000/chat et le terminal backend.

Partie B — État final et fermeture

Lire la fiche et prévoir le comportement

Ouvrez le **document fiche-reference**. Anticipez les deux observations qui suivront un clic sur Déconnecter : demande de fermeture, puis événement de fermeture.

```
information.value = 'TODO 1 : instancier WebSocket dans useConnection.js.';
return;
```

par la construction native affectée à **current**, en utilisant **url.value** comme adresse. Ne créez pas un client Socket.IO. Ne mettez pas **await** devant le constructeur.

Résultat intermédiaire : le navigateur peut tenter une ouverture, même si votre indicateur n'est pas encore synchronisé. Les autres TODO restent nécessaires.

TODO 5 · Traiter close

Complétez **onClose(event)** pour :

1. Recopier l'état courant du socket dans r**eadyState.value**.
2. Afficher une information de fermeture, éventuellement différente selon **event.wasClean**.
3. Appeler **record('close', détail)** avec **event.code**, **event.reason** et **event.wasClean** dans le détail.

Le journal doit conserver un motif vide comme une absence d'information, pas le transformer en « serveur arrêté ». À la fin, l'état vaut 3 et une nouvelle ouverture manuelle est possible.

Indice progressif : commencez par enregistrer le code ; ajoutez ensuite la raison et **wasClean** pour aider le diagnostic. Aucun décodage JSON n'est nécessaire.

TODO 6 · Fermer volontairement

Dans **disconnect()** :

1. Demandez une fermeture normale avec le code 1000 et une raison courte.
2. Copiez aussitôt l'état courant de socket dans **readyState.value**.
3. Conservez le message et la ligne de journal fournis.

N'affectez pas directement **CLOSED** pour faire disparaître le badge : attendez l'événement qui confirme la fermeture. N'utilisez aucun temporisateur.

Vérifier le parcours nominal

Connecter → **open** → Déconnecter → **close**. La transition 2 peut être visuellement très brève : le journal en garde la trace. Attendez ensuite quelques secondes ; aucune reconnexion ne doit se produire sans clic.

N'affectez pas directement **CLOSED** pour faire disparaître le badge : attendez l'événement qui confirme la fermeture. N'utilisez aucun temporisateur.

Bilan individuel

Répondez en une ou deux phrases :

1. Pourquoi faut-il attendre **open** pour afficher Connecté ?
2. Quelle différence faites-vous entre appeler **close()** et recevoir **close** ?
3. Pourquoi un événement **error** ne permet-il pas d'affirmer « le serveur est éteint » ?
4. Quelle preuve distingue la connexion du chat de celle de Vite ?

*Les réponses à ces questions sont disponibles dans le **document corrigés***

Critère de passage

Pour que le TP soit considéré comme valide, la connexion native doit s'ouvrir, son état doit être visible, une fermeture volontaire doit aboutir et les trois échecs demandés ne doivent pas produire de faux succès.

Dès lors que vous avez terminé le TP de cette séquence et bien compris comment ouvrir et gérer une connexion WebSocket, vous pourrez passer à la suite du cours.